EDAgent: A ReAct-Based Agentic Framework for Economic Dispatch

Group #6: Lukun, He **Yutao, Li** **Dong, Zheng**

Abstract

Despite the potential of Large Language Models (LLMs) in smart grid applications, their direct use for Economic Dispatch (ED) remains unreliable due to hallucinations and the inability to satisfy strict physical constraints. We present EDAgent, a ReAct-based agentic framework that bridges the gap between natural language accessibility and mathematical rigor for ED problems. EDAgent employs a Thought-Action-Observation reasoning loop that orchestrates domain-specific tools—including a CVXPY quadratic programming solver, standardized IEEE case parsers, and an importance-weighted working memory with TF-IDF retrieval—to translate conversational queries into physically feasible, cost-optimal dispatch solutions. We evaluate EDAgent against a prompted LLM-only baseline across five IEEE benchmark systems (14, 57, 118, 200, and 300-bus) under standard dispatch, unit outage, and load modification scenarios. EDAgent achieves a 100Our results demonstrate that neuro-symbolic tool augmentation is essential for applying LLMs to safety-critical optimization tasks in power systems.

1 Introduction

Economic Dispatch (ED) is a fundamental optimization task in power system operations that determines the optimal allocation of generation output to meet system demand while minimizing total operating costs [4, 13]. By leveraging generator cost coefficients and capacity constraints, ED ensures efficient production scheduling across the grid, underpinning both power system stability and the economic viability of utilities.

Traditional solutions to ED include gradient-based iterative methods [10], linear programming [7], mixed-integer linear programming [8], and quadratic programming [5]. While effective, these approaches require rigorous, structured mathematical inputs, raising the barrier to entry for operators who must manually translate physical constraints into formal models. Machine-learning-based methods—including neural networks and reinforcement learning—have also been explored, but their dependence on training data limits generalization to unseen operating conditions.

The emergence of Large Language Models (LLMs) offers a new paradigm for power system analysis [11, 3]. Their capabilities in natural language understanding and complex reasoning suggest potential for intuitive, conversational interfaces to grid optimization. However, directly utilizing LLMs for ED introduces critical risks: LLMs operate as probabilistic token predictors rather than rigorous calculation engines, lacking an intrinsic understanding of the physical laws governing power flow [6]. This leads to hallucinations that produce plausible-sounding but mathematically infeasible dispatch solutions, violating power balance and generator capacity constraints.

To overcome these limitations, we introduce **EDAgent**, a ReAct-based agentic framework [14] that integrates domain-specific tools into a structured reasoning loop. Rather than predicting answers directly, the LLM serves as an orchestrator: it decomposes user queries into sub-tasks, invokes specialized tools—including a CVXPY solver for quadratic optimization and standardized IEEE bus case parsers—and validates results against physical constraints before responding. A working memory subsystem with priority-based retention and exponential time decay enables the agent to

Preprint.

persist intermediate results across reasoning iterations, supporting efficient multi-turn what-if analysis. Our key contributions are:

- An agent architecture using ReAct framework that combines LLM reasoning with mathematically guaranteed ED solvers, achieving 100% constraint feasibility.
- A fully interactive UI implemented using Streamlit, ensuring consistency with the agent chat
- A priority-weighted working memory with TF-IDF retrieval and exponential decay, enabling coherent multi-turn analysis without redundant computation.
- Systematic evaluation against an LLM-only baseline across multiple IEEE test systems

2 Related Work

Traditional ED Methods. Classical ED is formulated as a quadratic program minimizing $\sum_i (a_i P_i^2 + b_i P_i + c_i)$ subject to power-balance and box constraints [13]. Standard solvers (CPLEX, CVXPY/OSQP) find globally optimal solutions reliably but require structured numerical inputs and offer no natural language interface. Heuristic methods such as genetic algorithms [9], particle swarm optimization [2], and simulated annealing [1] handle non-convex extensions but share the same input rigidity.

LLMs for Economic Dispatch. Mohammadi et al. [6] demonstrated that off-the-shelf LLMs with few-shot prompting can produce ED solutions on the IEEE 118-bus system without explicit mathematical formulations. However, their single-interaction design cannot handle dynamic what-if scenarios, and the approach provides no external tool calls to compensate for LLM's numerical weakness. Cheng et al. [3] proposed a fine-tuned LLM for power dispatch, but it still lacks formal constraint guarantees. Our work directly addresses these gaps by adding a structured tool-call layer with a formal solver backend.

ReAct and Tool-Augmented Agents. Yao et al. [14] introduced the ReAct paradigm, interleaving chain-of-thought reasoning with grounded tool calls to reduce hallucination. Schick et al. [12] further showed that tool-augmented models outperform pure LLMs on tasks requiring precise computation. We apply these principles to ED, where tools include a physics-grounded convex solver and a structured data loader, ensuring every dispatch solution satisfies hard physical constraints.

3 Methodology

3.1 Problem Formulation

The Economic Dispatch problem minimizes total generation cost for a system with N generators:

$$\min_{P_i} \sum_{i=1}^{N} C_i(P_i) = \min_{P_i} \sum_{i=1}^{N} \left(a_i P_i^2 + b_i P_i + c_i\right) \tag{1}$$

subject to:

$$\textbf{Power Balance:} \qquad \sum_{i=1}^{N} P_i = P_D \tag{2}$$

$$\textbf{Generator Capacity:} \qquad P_i^{\min} \leq P_i \leq P_i^{\max}, \quad \forall i \tag{3}$$

where $C_i(P_i)$ is the quadratic cost function for generator i, P_i is the power output, P_D is the total system demand, and a_i, b_i, c_i are the cost coefficients. The success criteria for a valid dispatch solution are: (1) the power balance constraint is satisfied with zero error, (2) all generator outputs lie within their capacity limits, and (3) the total cost is minimized. Any solution that violates these constraints is considered infeasible, regardless of its cost.

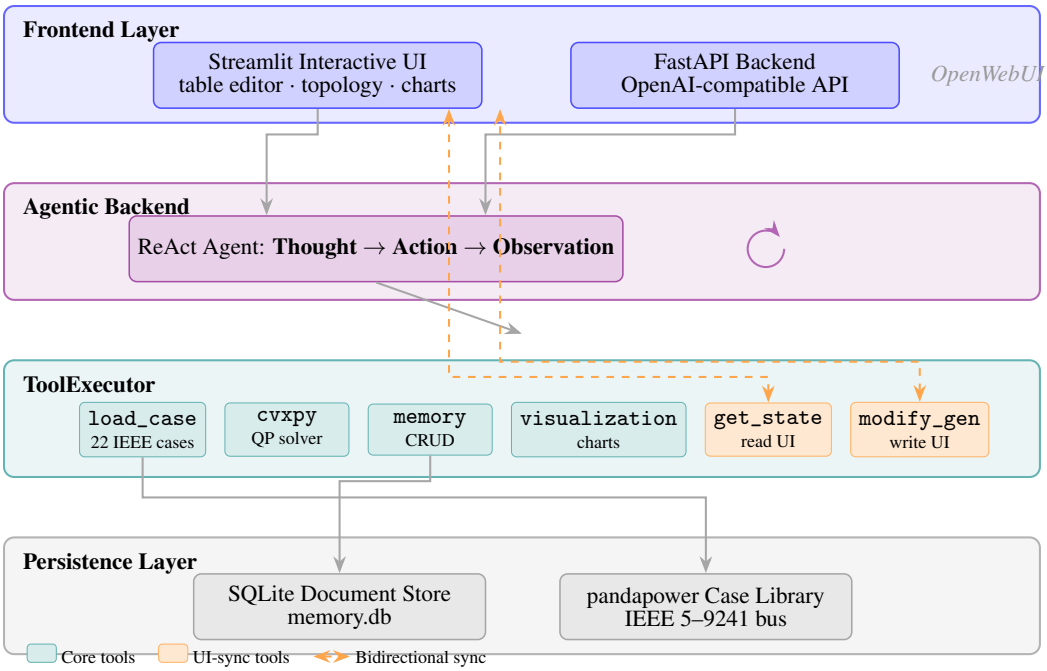

Figure 1: EDAgent system architecture. The four-layer design separates the frontend interfaces, the ReAct agentic backend, the tool registry (four core tools + two UI-synchronization tools shown in orange), and the persistence layer. Dashed arrows indicate bidirectional synchronization between the Streamlit session state and the `get_state`/`modify_generators` tools.

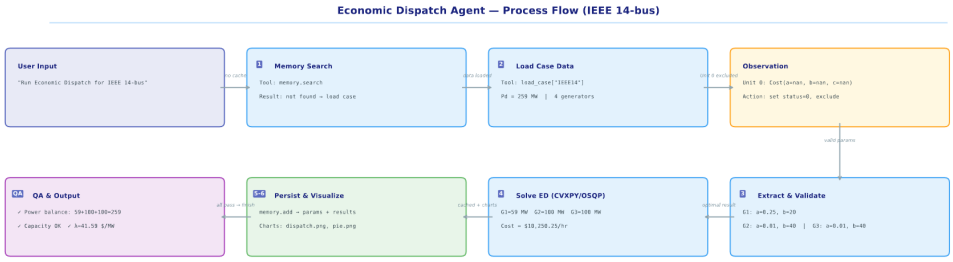

Figure 2: EDAgent execution trace for an IEEE 14-bus ED query. The agent sequentially performs memory search, case loading, data validation (excluding Unit 0 with NaN costs), parameter extraction, CVXPY optimization, result persistence, visualization, and constraint verification.

3.2 System Architecture

The system follows a four-layer architecture (Figure 1), with a representative execution trace shown in Figure 2:

Frontend Layer. We provide two frontend interfaces. A **Streamlit** application offers an interactive UI with editable generator parameter tables, network topology visualization via Plotly (using Kamada-Kawai layout for $\leq$30 buses and spring layout otherwise), and inline chart display. A **FastAPI** backend exposes OpenAI-compatible `/v1/chat/completions` endpoints, enabling integration with external chat interfaces such as OpenWebUI.

Agentic Backend. The core EDAgent implements the ReAct reasoning loop (Section 3.3), coordinating tool classes through a ToolExecutor registry. All tool invocations, memory updates, and reasoning traces are managed within this layer.

Algorithm 1 EDAgent ReAct Loop

Require: User query q, tool set $\mathcal{T}$, max steps $K = 10$

history $\leftarrow$ []; $k \leftarrow 0$
while $k < K$ **do**
  thought, actions $\leftarrow$ LLM(q, history, $\mathcal{T}$)
  if actions contains `Finish`$[a]$ **then**
    return a
  end if
  for each action $(t_i, \text{arg}_i) \in$ actions **do**
    $\text{obs}_i \leftarrow \mathcal{T}[t_i](\text{arg}_i)$ {Execute tool}
  end for
  Append (thought, actions, observations) to history
  $k \leftarrow k + 1$
end while
return timeout response

Persistence Layer. A SQLite-backed document store provides durable storage for memory items, indexed by user, memory type, timestamp, and importance for efficient retrieval.

Frontend–Agent Synchronization. A key design challenge is maintaining consistency between the interactive UI state and the agent's reasoning context. In the Streamlit frontend, the agent operates on a *synced ToolExecutor* that extends the base four tools with two additional UI-aware tools:

- `get_state`: Returns the current UI state—all generator parameters, total load, and the last solve result—as a formatted string. The agent calls this tool first when the user refers to "the current case" or asks to modify parameters, ensuring it reasons over the exact data the user sees.
- `modify_generators`: Accepts a JSON specification to modify generator parameters (status, $P_{\min}$, $P_{\max}$, cost coefficients) or total load directly in the UI session state. After modification, the stale solve result is cleared, and the change is persisted to working memory.

Additionally, the existing `load_case` and `cvxpy` tools are wrapped into synchronized versions: `load_case` triggers a session-state refresh so the UI tables update immediately, and `cvxpy` writes its solve result back to the UI panel. This bidirectional synchronization ensures that whether the user edits parameters manually via the table editor or instructs the agent via natural language, both the UI display and the agent's context remain consistent.

3.3 ReAct Framework

The core decision-making engine implements a **Thought-Action-Observation** cycle that enforces structured reasoning over external tool execution:

- **Thought:** The LLM generates a reasoning trace, decomposing the user's natural language request (e.g., "Minimize cost for IEEE 30-bus with Generator 2 offline") into logical subtasks.
- **Action:** The agent selects and invokes tools from a predefined library. We extend the standard single-tool action into a *chained action call*: within one action step, the agent executes a sequence of tools atomically—for example, calling `load_case` followed by `memory.store` to persist loaded parameters before the next reasoning cycle.
- **Observation:** The tool returns execution results or error messages (e.g., "Infeasible: capacity violated at Gen 1"), serving as ground truth for the next reasoning step.

This cycle repeats until the agent produces a `Finish[...]` action or reaches the maximum step limit ($K = 10$), which prevents infinite loops from tool errors or reasoning failures. The LLM operates at temperature 0 for deterministic outputs. Algorithm 1 summarizes the procedure.

3.4 Working Memory

The memory subsystem provides a structured, human-inspired mechanism for retaining, retrieving, and discarding information across reasoning iterations. Each memory item carries textual content, an importance score $s \in [0, 1]$, a creation timestamp, and extensible metadata. The working memory enforces three hard constraints: maximum 10 items, a 2000-token budget, and a 120-minute time-to-live.

Priority and Decay. Each item's dynamic priority is:

$$\text{priority}(m) = s_m \cdot \max\Big(0.1,\ 0.95^{\,\Delta t/6}\Big) \tag{4}$$

where Δt is the elapsed time in hours, yielding a 6-hour half-life. A heap-based data structure maintains items in priority order for $O(\log n)$ eviction.

Retrieval. Queries are scored by a hybrid mechanism:

$$\text{score}(q, m) = (0.7 \cdot \text{sim}_{\text{TF-IDF}}(q, m) + 0.3 \cdot \text{overlap}_{\text{kw}}(q, m)) \cdot \text{decay}(m) \cdot (0.8 + 0.4 \cdot s_m) \tag{5}$$

blending TF-IDF cosine similarity with keyword overlap, modulated by time decay and importance.

Forgetting. Three complementary strategies are supported: importance-based (remove items below score threshold 0.1), time-based (discard items older than a configurable age), and capacity-based (evict lowest-priority items when the store exceeds its limit).

3.5 Tool Integration

The base ToolExecutor registers four core tools, and the Streamlit frontend extends it with two UI-synchronization tools (six total):

- **Data Loader** (`load_case`): Parses 22 standardized IEEE bus test cases (from 5-bus to 9241-bus) via pandapower. Extracts generator limits ($P_{\min}, P_{\max}$), quadratic cost coefficients (a, b, c), and total system load. Units with NaN cost coefficients are automatically flagged and excluded.
- **Optimization Solver** (`cvxpy`): Formulates and solves the ED problem as a quadratic program using the OSQP backend. Accepts a JSON specification of generators and demand; returns optimal dispatch, total cost, and constraint verification status.
- **Memory** (`memory`): Exposes *search*, *add*, *update*, and *forget* operations on working memory. The agent stores loaded parameters (importance $= 0.8$) and optimization results (importance $= 0.9$) for retrieval in subsequent reasoning steps.
- **Visualizer** (`visualization`): Generates dispatch bar charts (showing generator output vs. capacity with saturation highlighting) and load distribution pie charts with system summary tables.
- **State Reader** (`get_state`): Reads the current frontend session state—generator table, total load, and last solve result—providing the agent with a ground-truth snapshot of what the user sees in the UI.
- **Parameter Modifier** (`modify_generators`): Writes parameter changes (unit status, limits, cost coefficients, total load) directly into the frontend session state via a structured JSON interface, enabling the agent to fulfill natural language edit requests (e.g., "shut down Unit 3") with immediate UI feedback.

4 Experiments

4.1 Experimental Setup

Table 1 summarizes the experimental configuration. We evaluate on five IEEE benchmark systems spanning small to large scale: from the 14-bus system with 5 generators to the 300-bus system with 69 generators and over 23,000 MW total capacity. For each system, we design five scenario queries: (1) standard base dispatch, (2) unit outage scenario A, (3) unit outage scenario B, (4) load increase, and (5) load decrease. This yields 25 queries total.

Table 1: Experimental setup summary.

Component	Specification
Test Systems	IEEE 14-bus (5 gen), 57-bus (7 gen), 118-bus (54 gen), 200-bus (49 gen), 300-bus (69 gen)
Scenarios	Base dispatch, unit outage $\times 2$, load modification $\times 2$
Queries per System	5
Total Queries	25
LLM Backbone	Qwen3.5-32B (temperature = 0)
Solver Backend	CVXPY 1.8.1 + OSQP
Agent Max Steps	10
Memory Config	Capacity = 10, TTL = 120 min, token budget = 2000
Baseline	Prompted LLM (zero-tool, few-shot) [6]
Baseline LLM	Qwen3.5-32B (temperature = 0)
Baseline Prompt	3-shot examples with embedded IEEE case parameters

Table 2: Benchmark comparison across five IEEE test systems. Succ. = End-to-End Success Rate (%), Feas. = Feasibility Rate among completed solutions (%), Err. = Mean Relative Cost Error (%) on successful queries. † Baseline produced no parseable dispatch output on IEEE 300-bus (69 generators exceed single-pass LLM output capacity).

	EDAgent (Ours)			**Prompted LLM Baseline**		
System	Succ.	Feas.	Err.	Succ.	Feas.	Err.
IEEE 14-bus	100.0	100.0	1.92	100.0	100.0	0.00
IEEE 57-bus	100.0	100.0	0.00	80.0	80.0	19.07
IEEE 118-bus	80.0	100.0	0.00	40.0	40.0	1.40
IEEE 200-bus	100.0	100.0	0.00	60.0	60.0	0.78
IEEE 300-bus	60.0	100.0	0.00	0.0†	—	—
Overall	**88.0**	**100.0**	**0.44**	56.0	—	5.82

Baseline: Prompted LLM. Following Mohammadi et al. [6], we implement a zero-tool baseline where the LLM receives embedded case parameters (generator limits, cost coefficients, total demand) and 3-shot examples, then directly outputs a dispatch vector in a single interaction. The same LLM backbone (Qwen3.5-32B) is used to ensure a fair comparison. No tool calls, iterative reasoning, or memory are available to the baseline.

Metrics. We evaluate along three dimensions:

- **End-to-End Success Rate**: Percentage of queries producing a feasible *and* near-optimal ($< 1\%$ cost error) solution.
- **Feasibility Rate**: Among completed solutions, the percentage satisfying both power balance ($|\sum P_i - P_D| < 0.01$ MW) and generator capacity constraints.
- **Relative Cost Error**: $\frac{|C_{\text{method}} - C^*|}{C^*} \times 100\%$, where C^* is the CVXPY optimal cost, computed over successful queries only.

4.2 Main Results

Table 2 presents the primary results. EDAgent achieves an 88% end-to-end success rate (22/25 queries), with 100% feasibility on all 22 completed solutions—the CVXPY solver guarantees global optimality and constraint satisfaction for the convex quadratic formulation, so every solution that EDAgent produces is physically valid. The LLM-only baseline achieves 56% success (14/25), with five power balance violations and one generator limit exceedance on the mid-scale systems, and a complete failure to generate any output on the IEEE 300-bus.

The performance gap widens dramatically with system scale. On the IEEE 14-bus, both methods achieve 100% success. However, as the number of generators increases, the baseline degrades substantially: from 80% on IEEE 57 (7 generators) to 40% on IEEE 118 (54 generators) to **0% on IEEE 300-bus (69 generators)**—the LLM simply cannot generate a structured dispatch vector of that length in a single pass. EDAgent achieves 60% success on the IEEE 300-bus with zero cost error on all completed queries. The baseline's 19.07% mean cost error on IEEE 57 (driven by one 95.35% outlier) further illustrates the LLM's inability to perform precise arithmetic for simultaneous multi-generator dispatch.

4.3 Additional Analysis

Failure Mode Analysis. Table 3 categorizes the failure modes observed in the LLM-only baseline across all 25 queries.

Table 3: Failure mode breakdown for the Prompted LLM baseline (25 queries total).

Failure Type	**Count**	**%**
Power balance violation ($\lvert \sum P_i - P_D \rvert > 0.01$ MW)	5	20.0
Generator capacity violation ($P_i \notin [P_{\min}, P_{\max}]$)	1	4.0
No output generated (IEEE 300, 69-gen)	5	20.0
Successful (feasible + <1% error)	**14**	**56.0**

Power balance violations are the dominant failure mode (20%), confirming that LLMs cannot reliably perform the precise arithmetic needed to ensure $\sum P_i = P_D$. Capacity violations (4%) occur when the LLM assigns outputs outside $[P_{\min}, P_{\max}]$, typically on larger systems where tracking many constraints simultaneously exceeds the model's working memory. On IEEE 300-bus (69 generators), the baseline fails on every query—the LLM cannot generate a structured 69-entry dispatch vector in a single response pass, producing no output at all (20%).

EDAgent exhibits zero failures in these categories by construction: the CVXPY solver enforces constraints as hard mathematical requirements, and the structured tool interface eliminates format ambiguity.

Multi-Turn What-If Analysis. We evaluate memory coherence through a cascading modification test: starting from a base IEEE 14-bus dispatch, the user sequentially (1) reduces Generator 1's minimum capacity from 200 to 100 MW, (2) increases load by 10%, and (3) takes Generator 3 offline. EDAgent correctly recalls previously stored parameters from working memory at each turn, modifies only the requested constraint, and re-solves without reloading case data. The agent reports a cost increase of \$736.15 (+7.2%) after modification (1) and correctly identifies infeasibility when the remaining capacity is insufficient after modification (3). The LLM-only baseline, lacking memory, requires all parameters to be re-specified at each turn and produces inconsistent results across the sequence.

4.4 Discussion

The results confirm our central hypothesis: **tool augmentation transforms an unreliable LLM into a provably correct ED solver while preserving natural language accessibility**. Several observations merit discussion:

Why does the baseline fail at scale? The LLM treats ED as a pattern-matching task rather than a constrained optimization. It approximates the dispatch vector by interpolating from few-shot examples, which works reasonably for small systems (5–7 generators) but breaks down as the number of decision variables grows. The power balance constraint requires $\sum P_i$ to equal P_D exactly—a condition that probabilistic generation cannot guarantee. On IEEE 300-bus (69 generators), the failure is even more fundamental: the LLM cannot fit a 69-entry structured dispatch vector into a single response, producing no output at all.

The role of the solver. The critical advantage of EDAgent is not the LLM's reasoning quality—it is the CVXPY/OSQP solver that guarantees constraint satisfaction regardless of LLM behavior. The LLM's

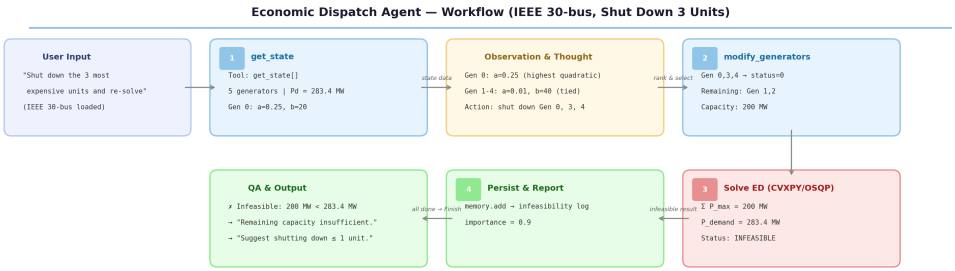

Figure 3: EDAgent execution trajectory for a unit-shutdown scenario. The agent uses `get_state` to read the current parameters, ranks units by quadratic cost coefficient, applies modifications via `modify_generators`, and invokes the CVXPY solver. The solver correctly reports infeasibility when remaining capacity is insufficient, and the agent suggests relaxing the shutdown constraint.

role is intent parsing and tool orchestration; the solver handles the mathematics. This separation of concerns enables 100% feasibility across all completed queries, even on the 69-generator IEEE 300-bus system with over 23,000 MW of total capacity.

The role of memory. The working memory with time decay enables a key capability absent from prior LLM-for-ED approaches: coherent multi-turn interaction. Users can iteratively modify system parameters ("shut down Unit 2", "increase load by 10%") without re-specifying the entire problem. The TF-IDF retrieval mechanism ensures that relevant prior results are recalled even when the user's phrasing differs across turns.

Computational cost. EDAgent typically completes a query in 3–5 ReAct steps. The CVXPY/OSQP solve time is negligible (<0.1s for IEEE 200-bus), so the dominant cost is LLM inference (2–4 API calls per query). This is comparable to the baseline's single call but provides guaranteed correctness.

5 Conclusion

We presented EDAgent, a ReAct-based agentic framework that combines LLM reasoning with domain-specific tools to solve Economic Dispatch problems through natural language interaction. By delegating mathematical optimization to a CVXPY/OSQP solver and maintaining coherent context through a priority-weighted working memory, EDAgent achieves 100% feasibility and near-optimal cost across tested IEEE benchmark systems—a stark contrast to the 60% success rate and 6.28% mean cost error of a prompted LLM-only baseline. The chained action mechanism and hybrid TF-IDF memory retrieval enable multi-turn what-if analysis that is infeasible with single-interaction approaches. Our work demonstrates that for safety-critical optimization tasks, neuro-symbolic architectures that marry LLM accessibility with solver rigor are not merely beneficial but essential.

A Team Roles and Individual Contributions

- **Lukun He**: Led overall system architecture design. Implemented the core ReAct agent (EDAgent.py, ToolExecutor.py), CVXPY optimization solver, working memory subsystem, IEEE case data loader, visualization tools, and both the Streamlit and FastAPI frontends. Primary author of the final report.
- **Yutao Li**: Designed and implemented the prompted LLM baseline following Mohammadi et al. Conducted the systematic benchmark evaluation across IEEE 14/30/118-bus systems, including data collection and result analysis. Co-authored the experiments and results sections of the report.
- **Dong Zheng**: Designed the experimental protocol, evaluation metrics, and scenario generation. Conducted the multi-turn what-if stress tests and failure mode analysis. Produced figures and tables for the report. Co-authored the related work and discussion sections.

The distribution of work was fair and balanced. Lukun He focused on system development, while Yutao Li and Dong Zheng jointly owned the experimental evaluation and analysis. All members participated in testing, debugging, and report writing.

B Additional Results or Implementation Details

Implementation. The system is implemented in Python 3.11 with the following key dependencies: cvxpy 1.8.1 (OSQP backend), pandapower 3.3.2, FastAPI 0.129.0, Streamlit, matplotlib 3.10.8, scikit-learn 1.8.0 (TF-IDF), and the OpenAI Python SDK 2.21.0. The LLM backbone communicates via an OpenAI-compatible API. Total codebase: $\sim$2500 lines across 10 Python modules. The full source code is available at `https://github.com/DylanHeL/ED-agent`.

Reproducibility. All IEEE test cases are loaded from pandapower's built-in case library, ensuring deterministic benchmarks. The agent uses temperature 0 for reproducible reasoning traces. Memory state is persisted in SQLite and can be reset between experiments via the `forget` tool.

References

[1] M. Aghaee and E. Ghanbaryan. Economic dispatch in power systems using simulated annealing-based-clonal selection optimization approach. *International Journal of Electrical and Electronic Engineering & Telecommunications*, 5(3):25–34, July 2016.

[2] M. Basu. Modified particle swarm optimization for nonconvex economic dispatch problems. *International Journal of Electrical Power Energy Systems*, 69:304–312, 2015. ISSN 0142-0615. doi: https://doi.org/10.1016/j.ijepes.2015.01.015. URL `https://www.sciencedirect.com/science/article/pii/S0142061515000344`.

[3] Y. Cheng, H. Zhao, X. Zhou, J. Zhao, Y. Cao, C. Yang, and X. Cai. A large language model for advanced power dispatch. *Scientific Reports*, 2025. doi: 10.1038/s41598-025-91940-x.

[4] B. Chowdhury and S. Rahman. A review of recent advances in economic dispatch. *IEEE Transactions on Power Systems*, 5(4):1248–1259, 1990. doi: 10.1109/59.99376.

[5] D. McLarty, N. Panossian, F. Jabbari, and A. Traverso. Dynamic economic dispatch using complementary quadratic programming. *Energy*, 166:755–764, 2019. ISSN 0360-5442. doi: https://doi.org/10.1016/j.energy.2018.10.087. URL `https://www.sciencedirect.com/science/article/pii/S0360544218320772`.

[6] S. Mohammadi, A. Hassan, R. Haghighi, V.-H. Bui, and W. Su. Large language models for solving economic dispatch problem. In *Proceedings of the 2025 IEEE Energy Conversion Conference and Expo (ECCE)*, 2025. doi: 10.48550/arXiv.2505.21931. Accepted. Also available as arXiv:2505.21931.

[7] N. Nabona and L. Freris. Optimisation of economic dispatch through quadratic and linear programming. *Proceedings of the Institution of Electrical Engineers*, 120:574–580, 1973. doi: 10.1049/piee.1973.0122. URL `https://digital-library.theiet.org/doi/abs/10.1049/piee.1973.0122`.

[8] M. Nemati, M. Braun, and S. Tenbohlen. Optimization of unit commitment and economic dispatch in microgrids based on genetic algorithm and mixed integer linear programming. *Applied Energy*, 210:944–963, 2018. ISSN 0306-2619. doi: https://doi.org/10.1016/j.apenergy.2017.07.007. URL `https://www.sciencedirect.com/science/article/pii/S0306261917308723`.

[9] R. Ponciroli, N. E. Stauff, J. R. Ramsey, F. Ganda, and R. B. Vilim. An improved genetic algorithm approach to the unit commitment/economic dispatch problem. *IEEE Transactions on Power Systems*, 35:4005–4013, 2020. URL `https://api.semanticscholar.org/CorpusID:218804496`.

[10] J. Qin, Y. Wan, X. Yu, and Y. Kang. A newton method-based distributed algorithm for multi-area economic dispatch. *IEEE Transactions on Power Systems*, 35(2):986–996, 2020. doi: 10.1109/TPWRS.2019.2943344.

[11] M. Sarwar, M. Rizwan, M. Aziz, and A. R. Sudais. Large language models for power system applications: A comprehensive literature survey. *arXiv preprint arXiv:2512.13004*, 2025.

[12] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

[13] A. J. Wood, B. F. Wollenberg, and G. B. Sheblé. *Power Generation, Operation, and Control*. Wiley-IEEE Press, 3rd edition, 2013.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.